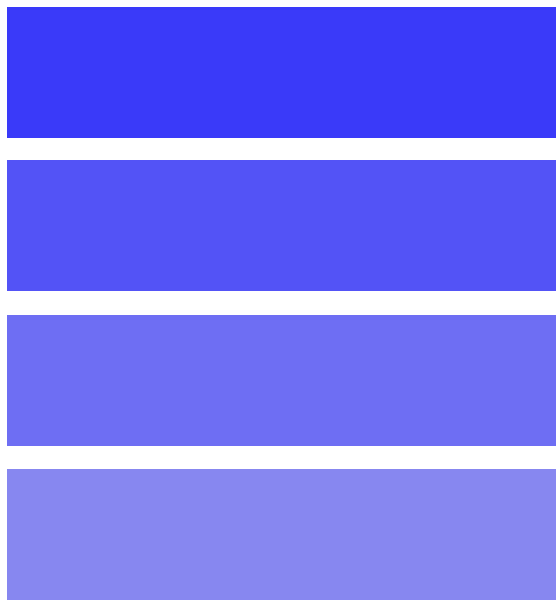


BÀI TẬP CƠ BẢN MẢNG 1 CHIỀU



NỘI DUNG (Kết hợp với contest mảng 1 chiều cơ bản trên hackerrank)

1. Liệt kê phần tử
2. Tìm min, max
3. Bài toán cặp số
4. Số Fibonacci & mảng 1 chiều
5. Mảng đối xứng, lật mảng
6. Liệt kê, đếm giá trị khác nhau bằng vòng lặp ($O(N^2)$)
7. Xóa phần tử
8. Chèn phần tử
9. Trộn 2 dãy tăng dần để tạo thành 1 dãy tăng dần
10. Mảng đánh dấu | Lùa bò vào chuồng
11. Xoay phải | Right rotate
12. Xoay trái | Left rotate
13. Dãy con liên tiếp
14. Truy vấn tổng tĩnh với mảng cộng dồn
15. Cặp số có tổng bằng K trong mảng tăng dần



1. Liệt kê phần tử thỏa mãn



Dạng bài tập này khá đơn giản, nó sẽ yêu cầu bạn liệt kê hoặc đếm các phần tử trong mảng thỏa mãn tính chất cho trước như : số nguyên tố, thuận nghịch, hoàn hảo....



Hướng giải : Bạn chỉ cần duyệt dãy số từ đầu tới cuối và gọi hàm để kiểm tra các phần tử trong mảng có thỏa mãn tính chất cần kiểm tra hay không ?

1. Liệt kê phần tử thỏa mãn



Ví dụ : Liệt kê các số nguyên tố trong mảng 1 chiều

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  bool nt(int n){
7      for(int i = 2; i <= sqrt(n); i++){
8          if(n % i == 0) return false;
9      }
10     return n > 1;
11 }
12
13 int main(){
14     int a[] = {3, 1, 2, 5, 9, 11, 18, 19, 17, 55, 40, 23};
15     int n = 12;
16     for(int i = 0; i < n; i++){
17         if(nt(a[i])){
18             cout << a[i] << " ";
19         }
20     }
21 }
```



2. Tìm min, max trong mảng



Để tìm số lớn nhất, nhỏ nhất trong dãy số bạn có thể sử dụng 2 cách phổ biến. Chỉ cần duyệt 1 lượt qua dãy số là có thể tìm được min, max



Hướng giải : Ban đầu bạn giả sử min (max) sẽ là phần tử đầu tiên trong mảng ($a[0]$), sau đó duyệt từ phần tử thứ 2 trong mảng trở đi và cập nhật giá trị cho min (max).

2. Tìm min, max trong mảng



Ví dụ : Tìm số nhỏ nhất, lớn nhất trong mảng 1 chiều

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  int main(){
7      int a[] = {3, 1, 2, 5, 9, 11, 18, 19, 17, 55, 40, 23};
8      int n = 12;
9      int min_val = a[0], max_val = a[0];
10     for(int i = 0; i < n; i++){
11         if(a[i] < min_val){
12             min_val = a[i];
13         }
14         if(a[i] > max_val){
15             max_val = a[i];
16         }
17     }
18     cout << min_val << " " << max_val << endl;
19 }
```



2. Tìm min, max trong mảng



Ví dụ : Tìm số nhỏ nhất, lớn nhất trong mảng 1 chiều cách 2

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  int main(){
7      int a[] = {3, 1, 2, 5, 9, 11, 18, 19, 17, 55, 40, 23};
8      int n = 12;
9      int min_val = INT_MAX, max_val = INT_MIN;
10     for(int i = 0; i < n; i++){
11         if(a[i] < min_val){
12             min_val = a[i];
13         }
14         if(a[i] > max_val){
15             max_val = a[i];
16         }
17     }
18     cout << min_val << " " << max_val << endl;
19 }
```



3. Liệt kê cặp số trong mảng



Dạng bài tập này thường yêu cầu bạn đếm (liệt kê) các cặp số trong mảng thỏa mãn tính chất cho trước như : nguyên tố cùng nhau, có tổng bằng K,



Hướng giải : Dùng 2 vòng for lồng nhau để xét tất cả các cặp, đối với mỗi chỉ số i trong mảng sẽ duyệt các chỉ số j từ $i + 1$ tới cuối mảng để xét cặp $a[i]$ và $a[j]$. **Độ phức tạp là $O(N^2)$**

3. Liệt kê cặp số trong mảng



Ví dụ : Đếm số cặp số có tổng bằng K trong mảng

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  int main(){
7      int a[] = {1, 2, 3, 4, 5, 6, 7, 8};
8      int n = 8, k = 9;
9      int dem = 0;
10     for(int i = 0; i < n; i++){
11         for(int j = i + 1; j < n; j++){
12             if(a[i] + a[j] == k){
13                 ++dem;
14             }
15         }
16     }
17     cout << dem << endl;
18 }
```



4. Số Fibonacci và mảng 1 chiều



Các bài tập liên quan tới dãy số Fibonacci hay các dãy số có công thức truy hồi bạn nên sử dụng mảng 1 chiều để lưu trữ và tính toán các số trong dãy cho tiện.



Hướng giải : Khai báo mảng $F[]$ đủ lớn trong đó $F[i]$ sẽ lưu số Fibonacci thứ i , khi cần tính $F[i]$ chỉ cần tính thông qua $F[i - 1]$ và $F[i - 2]$ là 2 phần tử đứng trước $F[i]$ trong mảng

4. Số Fibonacci và mảng 1 chiều



Ví dụ 1: Kiểm tra số N có phải là số trong dãy Fibonacci hay không ?

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  int main(){
7      long long F[100];
8      F[0] = 0; F[1] = 1;
9      for(int i = 2; i <= 92; i++){
10         F[i] = F[i - 1] + F[i - 2];
11     }
12     long long n; cin >> n;
13     for(int i = 0; i <= 92; i++){
14         if(F[i] == n){
15             cout << "YES\n"; return 0;
16         }
17     }
18     cout << "NO\n";
19 }
```



4. Số Fibonacci và mảng 1 chiều



Ví dụ 2: Liệt kê N số đầu tiên trong dãy số Fibonacci

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  int main(){
7      long long F[100];
8      F[0] = 0; F[1] = 1;
9      for(int i = 2; i <= 92; i++){
10         F[i] = F[i - 1] + F[i - 2];
11     }
12     int n; cin >> n;
13     for(int i = 0; i < n; i++){
14         cout << F[i] << " ";
15     }
16 }
17
```



5. Mảng đối xứng, lật ngược mảng



Bài tập này sẽ yêu cầu bạn kiểm tra mảng đã cho có phải là đối xứng (thuận nghịch) hay không, đôi khi là yêu cầu bạn lật ngược mảng



Hướng giải : So sánh phần tử đối xứng với nhau trong mảng, nếu 2 phần tử này khác nhau thì kết luận mảng không đối xứng. Các bạn có thể dùng 2 chỉ số left, right để xét cặp. **Độ phức tạp $O(N)$**



5. Mảng đối xứng, lật ngược mảng



Ví dụ 1: Kiểm tra mảng đối xứng (2 cách)

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  bool check(int a[], int n){
7      int left = 0, right = n - 1;
8      while(left <= right){
9          if(a[left] != a[right]){
10             return false;
11         }
12         ++left; --right;
13     }
14     return true;
15 }
16
17 int main(){
18     int a[] = {1, 2, 3, 3, 2, 1};
19     if(check(a, 6)) cout << "YES\n";
20     else cout << "NO\n";
21 }
```

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  bool check(int a[], int n){
7      for(int i = 0; i < n / 2; i++){
8          if(a[i] != a[n - i - 1])
9              return false;
10     }
11     return true;
12 }
13
14 int main(){
15     int a[] = {1, 2, 3, 3, 2, 1};
16     if(check(a, 6)) cout << "YES\n";
17     else cout << "NO\n";
18 }
19
20
21
```



5. Mảng đối xứng, lật ngược mảng



Ví dụ 2: Lật ngược mảng (2 cách)

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  void reverse(int a[], int n){
7      int left = 0, right = n - 1;
8      while(left <= right){
9          swap(a[left], a[right]);
10         ++left; --right;
11     }
12 }
13
14 int main(){
15     int a[] = {1, 2, 3, 4, 5, 6};
16     reverse(a, 6);
17     for(int x : a) cout << x << " ";
18 }
19
20
```

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  void reverse(int a[], int n){
7      for(int i = 0; i < n / 2; i++){
8          swap(a[i], a[n - i - 1]);
9      }
10 }
11
12 int main(){
13     int a[] = {1, 2, 3, 4, 5, 6};
14     reverse(a, 6);
15     for(int x : a) cout << x << " ";
16 }
17
18
```



6. Liệt kê, đếm giá trị khác nhau



Dạng bài tập này thường liên quan tới đếm, liệt kê các số khác nhau trong mảng, hoặc đếm tần suất



Hướng giải : Ở thời điểm hiện tại các bạn sẽ dùng thuật toán ngây thơ là 2 vòng for lồng nhau, sau này sẽ tối ưu với set map hoặc sort.
Độ phức tạp $O(N^2)$

6. Liệt kê, đếm giá trị khác nhau



Ví dụ 1: Liệt kê các giá trị khác nhau trong mảng theo thứ tự xuất hiện



Ý tưởng : Đối với mỗi chỉ số i trong mảng ta xét các chỉ số từ 0 tới $i - 1$ để xét xem đã có phần tử giống $a[i]$ được in ra hay chưa, nếu không tồn tại thì sẽ in ra $a[i]$ vì đây là lần đầu tiên gặp giá trị này

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  int main(){
7      int a[] = {1, 5, 1, 1, 2, 3, 1, 4, 5, 0};
8      int n = 10;
9      for(int i = 0; i < n; i++){
10         bool found = false;
11         for(int j = 0; j < i; j++){
12             if(a[i] == a[j]){
13                 found = true; break;
14             }
15         }
16         if(!found){
17             cout << a[i] << " ";
18         }
19     }
20 }
```



6. Liệt kê, đếm giá trị khác nhau



Ví dụ 2: Đếm số giá trị khác nhau trong mảng



Ý tưởng : Đối với mỗi chỉ số i trong mảng ta xét các chỉ số từ 0 tới $i - 1$ để xét xem đã có phần tử giống $a[i]$ được in ra hay chưa, nếu không tồn tại thì sẽ in ra $a[i]$ vì đây là lần đầu tiên gặp giá trị này ta sẽ tính nó là 1 giá trị khác nhau trong mảng

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  int main(){
7      int a[] = {1, 5, 1, 1, 2, 3, 1, 4, 5, 0};
8      int n = 10;
9      int dem = 0;
10     for(int i = 0; i < n; i++){
11         bool found = false;
12         for(int j = 0; j < i; j++){
13             if(a[i] == a[j]){
14                 found = true; break;
15             }
16         }
17         if(!found){
18             ++dem;
19         }
20     }
21     cout << dem << endl;
22 }
```



6. Liệt kê, đếm giá trị khác nhau



Ví dụ 3: Đếm tần suất của mỗi số trong mảng và liệt kê



Ý tưởng : Tương tự như 2 ví dụ ở trên đối với $a[i]$ ta sẽ xét xem đã gặp $a[i]$ trước đó hay chưa, nếu chưa gặp thì sẽ tiến hành đếm tần suất của nó và in ra. Ta sử dụng 1 vòng for để check xem đã gặp $a[i]$ chưa và 1 vòng for khác để đếm tần suất.

```
1 #include <iostream>
2 #include <math.h>
3
4 using namespace std;
5
6 int main(){
7     int a[] = {1, 5, 1, 1, 2, 3, 1, 4, 5, 2};
8     int n = 10;
9     for(int i = 0; i < n; i++){
10         bool found = false;
11         for(int j = 0; j < i; j++){
12             if(a[i] == a[j]){
13                 found = true; break;
14             }
15         }
16         if(!found){
17             int ts = 1; // a[i]
18             for(int j = i + 1; j < n; j++){
19                 if(a[i] == a[j]) ++ts;
20             }
21             cout << a[i] << " xuất hiện " << ts << " lần\n";
22         }
23     }
24 }
```



7. Xóa phần tử trong mảng



Thực tế việc xóa phần tử trong mảng 1 chiều C++ chỉ mang tính chất tương đối, tức là bạn sẽ dịch các phần tử của mảng sang trái và giảm kích cỡ của mảng chứ số lượng ô nhớ của mảng là không đổi



Hướng giải : Khi xóa phần tử ở chỉ số K trong mảng, bạn cần dịch các phần tử từ chỉ số K + 1 tới cuối dãy sang trái 1 ô. Khi đó giá trị tại chỉ số K sẽ bị ghi đè. Cuối cùng thì giảm số lượng phần tử trong mảng đi 1 đơn vị.

7. Xóa phần tử trong mảng



Ví dụ : Xóa phần tử khỏi chỉ số k trong mảng



```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  void xoa(int a[], int &n, int k){
7      for(int i = k; i < n - 1; i++){
8          a[i] = a[i + 1];
9      }
10     --n;
11 }
12
13 int main(){
14     int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
15     int n = 10, k = 4;
16     xoa(a, n, k);
17     for(int i = 0; i < n; i++){
18         cout << a[i] << " ";
19     }
20 }
```



8. Chèn phần tử vào mảng



Để chèn thêm 1 phần tử vào trong mảng bạn cần phải đảm bảo kích thước của mảng sẽ đủ sau khi chèn. Thông thường bạn cần khai báo kích thước của mảng lớn hơn ít nhất 1 phần tử so với mảng ban đầu



Hướng giải : Để chèn phần tử X và chỉ số K trong mảng đầu tiên bạn cần dịch các phần tử từ chỉ số K tới cuối dãy sang phải 1 vị trí. Bạn phải dịch từ cuối về nếu không sẽ bị ghi đè. Sau đó thay $A[K] = X$ và tăng kích cỡ của mảng

8. Chèn phần tử vào mảng



Ví dụ : Chèn phần tử có giá trị X vào chỉ số K trong mảng



```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  void chen(int a[], int &n, int k, int x){
7      for(int i = n - 1; i >= k; i--){
8          a[i + 1] = a[i];
9      }
10     a[k] = x;
11     ++n;
12 }
13
14 int main(){
15     int a[100] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
16     int n = 10, k = 4;
17     chen(a, n, k, 28);
18     for(int i = 0; i < n; i++){
19         cout << a[i] << " ";
20     }
21 }
```



9. Trộn 2 dãy đã sắp xếp (merge)



Đề bài cho bạn 2 dãy $A[]$ và $B[]$ đã được sắp xếp tăng dần (giảm dần) và yêu cầu bạn trộn 2 dãy này lại mà vẫn thu được 1 dãy tăng dần ???



Hướng giải : Dùng 2 chỉ số i cho mảng $A[]$ và j cho mảng $B[]$, ở mỗi lần lặp thì so sánh $A[i]$ với $B[j]$ để biết phần tử nào nhỏ hơn thì sẽ in ra phần tử này trước cho tới khi 1 trong 2 dãy duyệt xong. Dãy còn lại còn phần tử sẽ được in ra tiếp. **Độ phức tạp $O(N + M)$**

9. Trộn 2 dãy đã sắp xếp (merge)



Ví dụ : In ra dãy số sau khi trộn 2 dãy tăng dần

```
1  #include <iostream>
2  #include <math.h>
3
4  using namespace std;
5
6  int main(){
7      int a[] = {1, 1, 3, 5, 8, 9};
8      int b[] = {2, 2, 3, 4, 9, 11, 14};
9      int n = 6, m = 7;
10     int i = 0, j = 0;
11     while(i < n && j < m){
12         if(a[i] <= b[j]){
13             cout << a[i] << " "; ++i;
14         }
15         else{
16             cout << b[j] << " "; ++j;
17         }
18     }
19     while(i < n){
20         cout << a[i] << " "; ++i;
21     }
22     while(j < m){
23         cout << b[j] << " "; ++j;
24     }
25 }
```



10. Mảng đánh dấu (đếm phân phối)



Kỹ thuật đếm phân phối hay mình hay gọi là mảng đánh dấu sử dụng cho những bài tập liên quan tới giá trị khác nhau, đếm tần suất, sàng số nguyên tố



Ý tưởng : Sử dụng một mảng đủ lớn ($10^6 - 10^7$) và sử dụng chỉ số của mảng này để đánh dấu sự xuất hiện của giá trị tương ứng. **Vì thế kỹ thuật này chỉ phù hợp với mảng chứa các số không âm và không được vượt quá chỉ số của mảng.**



10. Mảng đánh dấu (đếm phân phối)



Bài toán 1 : Đếm số lượng giá trị khác nhau trong mảng, các phần tử trong mảng nằm trong đoạn $[0, 10^6]$



Hướng giải : Khai báo mảng `cnt[ ]` đủ lớn để có thể đánh dấu được tới giá trị 10^6 , bạn cần 1 mảng đánh dấu cỡ ít nhất $10^6 + 1$ phần tử. Ban đầu các phần tử trong mảng này đều có giá trị bằng 0. Khi gặp giá trị X trong mảng thì bạn cho `cnt[X] = 1`.



10. Mảng đánh dấu (đếm phân phối)



Bài toán 1 : Đếm số lượng giá trị khác nhau trong mảng, các phần tử trong mảng nằm trong đoạn $[0, 10^6]$



Hướng giải : Khai báo mảng `cnt[ ]` đủ lớn để có thể đánh dấu được tới giá trị 10^6 , bạn cần 1 mảng đánh dấu cỡ ít nhất $10^6 + 1$ phần tử. Ban đầu các phần tử trong mảng này đều có giá trị bằng 0. Khi gặp giá trị X trong mảng thì bạn cho `cnt[X] = 1`.

10. Mảng đánh dấu (đếm phân phối)



Bài toán 1 : Đếm số giá trị khác nhau trong mảng

```
1  #include <iostream>
2  using namespace std;
3
4  int cnt[1000001];
5
6  int main(){
7      int a[10] = {1, 2, 3, 1, 1, 2, 0, 3, 5, 8};
8      int max_val = INT_MIN;
9      for(int i = 0; i < 10; i++){
10         cnt[a[i]] = 1; // đánh dấu
11         max_val = max(max_val, a[i]);
12     }
13     int ans = 0;
14     for(int i = 0; i <= max_val; i++){
15         if(cnt[i] == 1){
16             ++ans;
17         }
18     }
19     cout << ans << endl;
20 }
```



10. Mảng đánh dấu (đếm phân phối)



Bài toán 2 : Liệt kê giá trị khác nhau trong mảng theo thứ tự tăng dần

```
1  #include <iostream>
2  using namespace std;
3
4  int cnt[1000001];
5
6  int main(){
7      int a[10] = {1, 2, 3, 1, 1, 2, 0, 3, 5, 8};
8      int max_val = INT_MIN;
9      for(int i = 0; i < 10; i++){
10         cnt[a[i]] = 1; // đánh dấu
11         max_val = max(max_val, a[i]);
12     }
13     for(int i = 0; i <= max_val; i++){
14         if(cnt[i] == 1){
15             cout << i << " ";
16         }
17     }
18 }
19
```



10. Mảng đánh dấu (đếm phân phối)



Bài toán 3 : Liệt kê giá trị khác nhau trong mảng theo thứ tự xuất hiện

```
1  #include <iostream>
2  using namespace std;
3
4  int cnt[1000001];
5
6  int main(){
7      int a[10] = {1, 2, 3, 1, 1, 2, 0, 3, 5, 8};
8      for(int i = 0; i < 10; i++){
9          cnt[a[i]] = 1; // đánh dấu
10     }
11     for(int i = 0; i < 10; i++){
12         if(cnt[a[i]] != 0){
13             cout << a[i] << " ";
14             cnt[a[i]] = 0; // reset
15         }
16     }
17 }
18
19
```



10. Mảng đánh dấu (đếm phân phối)



Bài toán 4 : Đếm tần suất xuất hiện của các phần tử trong mảng

```
1  #include <iostream>
2  using namespace std;
3
4  int cnt[1000001];
5
6  int main(){
7      int a[10] = {1, 2, 3, 1, 1, 2, 0, 3, 5, 8};
8      int max_val = INT_MIN;
9      for(int i = 0; i < 10; i++){
10         cnt[a[i]]++; // đánh dấu
11         max_val = max(max_val, a[i]);
12     }
13     for(int i = 0; i <= max_val; i++){
14         if(cnt[i] != 0){
15             cout << i << " xuất hiện " << cnt[i] << " lần\n";
16         }
17     }
18 }
19
20
```



11. Xoay phải (Right rotate)



Bài toán này yêu cầu các bạn dịch phải các phần tử trong mảng của bạn sang 1 vị trí, phần tử cuối cùng trong mảng sẽ quay vòng lại vị trí đầu tiên : VD 1 2 3 4 5 -> 5 1 2 3 4



Hướng giải : Lưu lại giá trị của phần tử cuối cùng trong mảng, sau đó dịch các phần tử ngoại trừ phần tử cuối sang phải 1 vị trí, gán phần tử đầu tiên trong mảng bằng phần tử cuối đã lưu lúc trước. Lưu ý là nếu mảng có N phần tử thì sau N lần quay phải mảng sẽ về ban đầu.



11. Xoay phải (Right rotate)



In ra mảng sau khi dịch phải 1 phần tử



```
1  #include <iostream>
2  using namespace std;
3
4  void rightRotate(int a[], int n){
5      int tmp = a[n - 1];
6      for(int i = n - 1; i > 0; i--){
7          a[i] = a[i - 1];
8      }
9      a[0] = tmp;
10 }
11
12 int main(){
13     int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
14     rightRotate(a, 10);
15     for(int i = 0; i < 10; i++){
16         cout << a[i] << " ";
17     }
18 }
19
20
```



12. Xoay trái (Left rotate)



Bài toán này yêu cầu các bạn dịch trái các phần tử trong mảng của bạn sang 1 vị trí, phần tử đầu tiên trong mảng sẽ quay vòng lại vị trí cuối cùng: VD 1 2 3 4 5 -> 2 3 4 5 1



Hướng giải : Lưu lại giá trị của phần tử đầu tiên trong mảng, sau đó dịch các phần tử ngoại trừ phần tử đầu sang trái 1 vị trí, gán phần tử cuối trong mảng bằng phần tử đầu đã lưu lúc trước. Lưu ý là nếu mảng có N phần tử thì sau N lần quay trái mảng sẽ về như ban đầu.



12. Xoay trái (Left rotate)



In ra mảng sau khi dịch trái 1 phần tử



```
1  #include <iostream>
2  using namespace std;
3
4  void leftRotate(int a[], int n){
5      int tmp = a[0];
6      for(int i = 0; i < n - 1; i++){
7          a[i] = a[i + 1];
8      }
9      a[n - 1] = tmp;
10 }
11
12 int main(){
13     int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
14     leftRotate(a, 10);
15     for(int i = 0; i < 10; i++){
16         cout << a[i] << " ";
17     }
18 }
19
20
```



13. Dãy con liên tiếp



Bài toán này thường liên quan tới tìm dãy con liên tiếp như dãy con liên tiếp các số giống nhau dài nhất, dãy con tăng dài nhất, dãy con liên tiếp các phần tử là số nguyên tố...



Hướng giải : Đối với các bài toán dạng này thì bạn chỉ cần dùng 1 vòng lặp duyệt qua mảng và so sánh phần tử $a[i]$ với $a[i - 1]$. Tuy nhiên cần phải hết sức chú ý tới phần tử đầu tiên và dãy con kết thúc tại chỉ số cuối cùng trong mảng

13. Dãy con liên tiếp – Kiểm tra mảng tăng (giảm)



Kiểm tra mảng tăng dần (giảm dần)



```
1  #include <iostream>
2  using namespace std;
3
4  bool tang(int a[], int n){
5      for(int i = 1; i < n; i++){
6          if(a[i] < a[i - 1])
7              return false;
8      }
9      return true;
10 }
11
12 int main(){
13     int a[6] = {1, 1, 2, 3, 4, 6};
14     if(tang(a, 6)){
15         cout << "YES\n";
16     }
17     else{
18         cout << "NO\n";
19     }
20 }
```



13. Dãy con liên tiếp – Dãy con nguyên tố dài nhất



Tìm độ dài dãy con liên tiếp tăng chặt dài nhất, nếu có nhiều dãy con thì chọn dãy có tổng lớn nhất và xuất hiện đầu tiên



```
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      int a[13] = {1, 2, 3, 2, 5, 1, 7, 8, 9, 2, 3, 4, 5};
6      int n = 13;
7      int start = 0, len = 1, max_len = 1;
8      int tong = a[0], max_tong = a[0];
9      for(int i = 1; i < n; i++){
10         if(a[i] > a[i - 1]){
11             ++len; tong += a[i];
12         }
13         else{
14             if(len > max_len){
15                 max_len = len; max_tong = tong;
16                 start = i - max_len;
17             }
18             else if(tong > max_tong){
19                 max_tong = tong; start = i - max_len;
20             }
21             len = 1; tong = a[i];
22         }
23     }
```

```
24         if(len > max_len){
25             max_len = len; max_tong = tong;
26             start = n - max_len;
27         }
28         else if(tong > max_tong){
29             max_tong = tong; start = n - max_len;
30         }
31         cout << max_len << endl;
32         for(int i = 0; i < max_len; i++){
33             cout << a[start + i] << " ";
34         }
35     }
36
37
38
```



13. Dãy con liên tiếp – Dãy con giống nhau dài nhất



Tìm độ dài dãy con liên tiếp các phần tử đều giống nhau dài nhất, nếu có nhiều dãy thỏa mãn thì lấy dãy xuất hiện đầu tiên



```
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      int a[13] = {1, 2, 3, 2, 2, 2, 7, 9, 9, 9, 3, 4, 5};
6      int n = 13;
7      int start = 0, len = 1, max_len = 1;
8      for(int i = 1; i < n; i++){
9          if(a[i] == a[i - 1]){
10             ++len;
11          }
12          else{
13              if(len > max_len){
14                  max_len = len;
15                  start = i - max_len;
16              }
17              len = 1;
18          }
19      }
```

```
20  if(len > max_len){
21      max_len = len;
22      start = n - max_len;
23  }
24  cout << max_len << endl;
25  for(int i = 0; i < max_len; i++){
26      cout << a[start + i] << " ";
27  }
28 }
```



14. Mảng cộng dồn



Bài toán này yêu cầu bạn truy vấn tổng tính trên đoạn $[L, R]$ trong mảng. Có nhiều truy vấn và bạn cần tối ưu độ phức tạp cho mỗi truy vấn



Hướng giải : Cách làm thông thường đó là với mỗi truy vấn bạn sẽ duyệt từ chỉ số L tới R và cộng các phần tử này lại với nhau, độ phức tạp sẽ là $O(N)$, sử dụng mảng cộng dồn bạn có thể tối ưu xuống còn $O(1)$ cho mỗi truy vấn

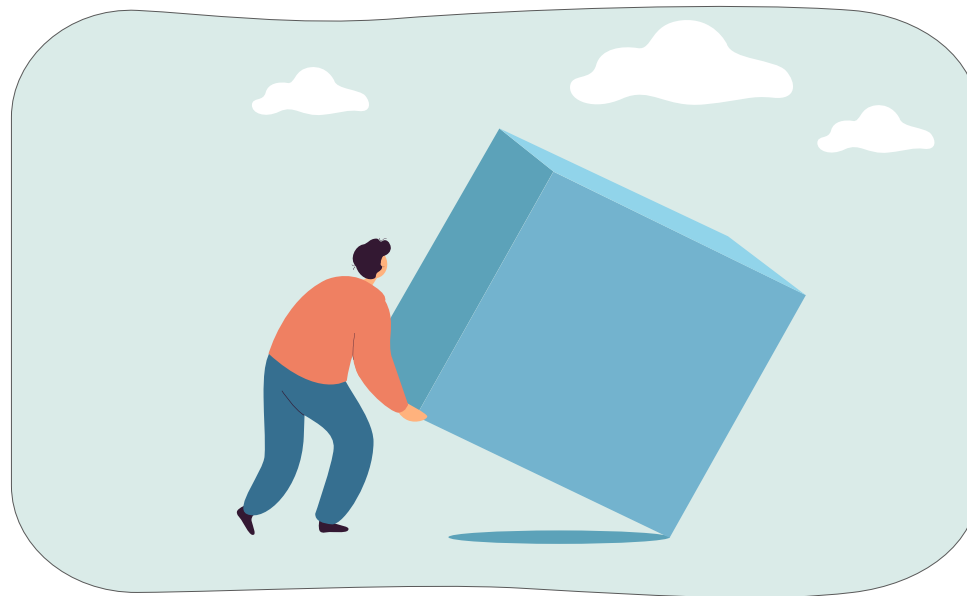


14. Mảng cộng dồn

Đặt vấn đề: Cho mảng $A[]$ có N phần tử, có Q truy vấn, mỗi truy vấn yêu cầu bạn tính tổng các phần tử từ chỉ số left tới chỉ số right.

Cách giải thông thường với mỗi truy vấn ($O(n)$)

```
int left, right;
cin >> left >> right;
int sum = 0;
for(int i=left; i<=right; i++){
    sum += A[i];
}
```



14. Mảng cộng dồn

Dùng mảng cộng dồn với mỗi truy vấn ($O(1)$)

Gọi mảng `pre[ ]` là mảng cộng dồn của mảng `A[ ]`.

Khi đó $pre[i] = A[0] + A[1] + A[2] + \dots + A[i]$ sẽ lưu tổng các phần tử từ chỉ số 0 tới chỉ số `i` của mảng `A[ ]`

Ta có thể `pre[i]` thông qua `pre[i - 1]` : $pre[i] = pre[i - 1] + A[i]$



Xây dựng mảng cộng dồn ($O(n)$)

```
int pre[n];
for(int i = 0; i < n; i++){
    if(i == 0)
        pre[i] = A[i];
    else
        pre[i] = pre[i - 1] + A[i];
}
```


14. Mảng cộng dồn



Để tính tổng các phần tử từ chỉ số left tới chỉ số right ta lấy $pre[right] - pre[left - 1]$, chú ý trường hợp $left = 0$.

A[]	4	2	3	1	5	6
Pre[]	4	6	9	10	15	21

Ví dụ áp dụng

EXAMPLE

```
int left, right;
cin >> left >> right;
if(left == 0){
    cout << pre[right] << endl;
}
else{
    cout << pre[right] - pre[left - 1] << endl;
}
```



15. Cặp số có tổng bằng K trong mảng tăng dần

Đặt vấn đề: Cho mảng $A[]$ có N phần tử đã được sắp xếp tăng dần, bạn hãy kiểm tra xem có tồn tại $A[i] + A[j] = K$?

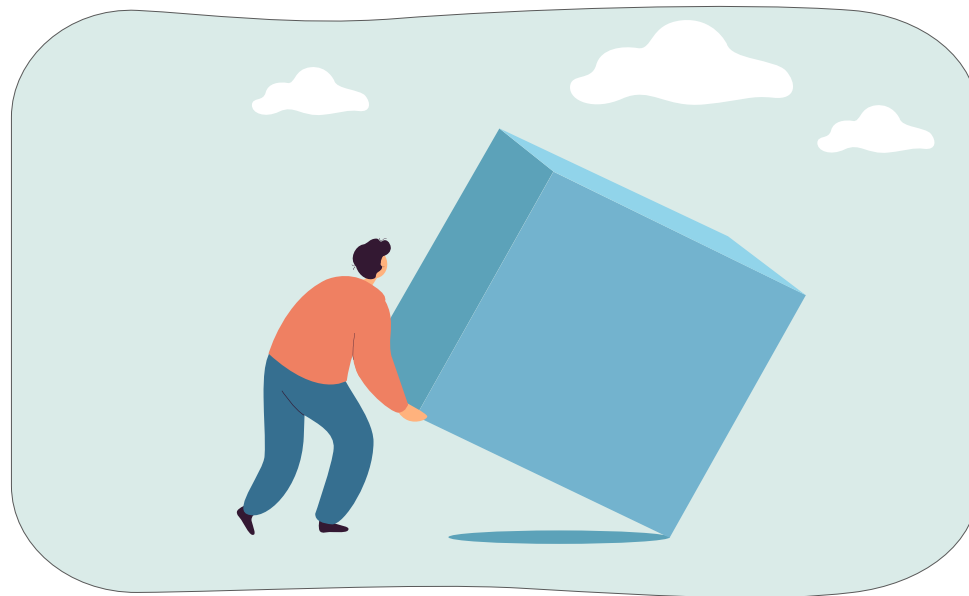


Bài toán này bạn sẽ sử dụng 2 chỉ số L, R tương ứng với phần tử đầu tiên và cuối cùng trong mảng. Khi so sánh $A[L] + A[R]$ với K sẽ có 3 trường hợp.

$A[L] + A[R] = K \Rightarrow$ Tìm thấy cặp

$A[L] + A[R] < K \Rightarrow$ Tăng L

$A[L] + A[R] > K \Rightarrow$ Giảm R



15. Cặp số có tổng bằng K trong mảng tăng dần



```
1  #include <iostream>
2  using namespace std;
3
4  bool check(int a[], int n, int K){
5      int l = 0, r = n - 1;
6      while(l < r){
7          if(a[l] + a[r] == K){
8              return true;
9          }
10         else if(a[l] + a[r] < K)
11             ++l;
12         else --r;
13     }
14     return false;
15 }
16
17 int main(){
18     int a[13] = {1, 2, 2, 2, 3, 3, 5, 7, 8, 9, 13, 23, 25};
19     int n = 13, K = 16;
20     cout << check(a, n, K) << endl;
21 }
```

